

蓝桥杯国赛JavaB组2022

2024-7-06 · marshall

本文记录了2022蓝桥杯国赛JavaB组的部分题解

1. 重合次数

```
import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println(494);
    }
}
```

2. 数数

```
import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);
    public static long get_divide(int x){
        long res = 0;
        for(int i = 2; i <= x / i ; i++) {
            if (x % i == 0) {
                int s = 0;
                while(x % i == 0) {
                    x /= i;
                    s++;
                }
                res += s;
            }
        }

        if (x > 1) res += 1;
        return res;
    }

    public static void main(String[] args) {
        int cnt = 0;
        for(int i = 2333333; i <= 233333333; i++) {
            if (get_divide(i) == 12) cnt++;
        }
        System.out.println(cnt);
    }
}
```

```

}
//25606
import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println(25606);
    }
}

```

3. 左移右移

```

import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);
    static int N = 400010,idx;
    static int[] e = new int[N], l = new int[N], r = new int[N];
    static int[] loc = new int[N];
    public static void init() {
        l[1] = 0;
        r[0] = 1;
        idx = 2;
    }

    public static void insert(int k, int x) {
        loc[x] = idx;
        e[idx] = x;
        l[idx] = k;
        r[idx] = r[k];
        l[r[k]] = idx;
        r[k] = idx++;
    }

    public static void remove(int k) {
        l[r[k]] = l[k];
        r[l[k]] = r[k];
    }

    public static void main(String[] args) {
        int n = sc.nextInt();
        int m = sc.nextInt();
        init();
        for(int i = 1; i <= n; i++) {
            insert(l[1],i);
        }
        while(m-->0) {
            String op = sc.next();
            int x = sc.nextInt();
            if (op.equals("L")) {
                remove(loc[x]);
                insert(0,x);
            }else {
                remove(loc[x]);
                insert(l[1],x);
            }
        }
    }
}

```

```

        for(int i = r[0]; i != 1; i = r[i]) System.out.print(e[i] + " ");

    }
}

```

4. 窗口

```

import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);
    static int N = 260,n,m,k;
    static char[][] g = new char[N][N];
    static HashMap<Integer, Pair> map = new HashMap<>();
    public static void main(String[] args) {
        n = sc.nextInt();
        m = sc.nextInt();
        k = sc.nextInt();
        for(int i = 0; i < N; i++) Arrays.fill(g[i], '.');
        for(int i = 0; i < k; i++) {
            String op = sc.next();
            if (op.equals("new")) {
                map.put(sc.nextInt(),
                        new Pair(sc.nextInt(), sc.nextInt(),
sc.nextInt(), sc.nextInt(),i));
            }else if (op.equals("resize")) {
                map.get(sc.nextInt()).resize(sc.nextInt(), sc.nextInt(), i);
            }else if (op.equals("move")) {
                map.get(sc.nextInt()).move(sc.nextInt(), sc.nextInt(), i);
            }else if (op.equals("close")) {
                map.get(sc.nextInt()).close();
            }else if(op.equals("active")){
                map.get(sc.nextInt()).active(i);
            }
        }
        List<Pair> list = new ArrayList<>();
        list.addAll(map.values());
        Collections.sort(list);
        for(Pair e: list) {
            int x1 = e.x,y1=e.y,x2=e.x+e.a-1,y2=e.y+e.b-1;
            for(int i = x1; i <= x2; i++) {
                for(int j = y1; j <= y2; j++) {
                    if (i >= 0 && i < n && j >= 0 && j < m) {
                        if((i == x1 || i == x2) && (j == y1 || j == y2))

                            g[i][j] = '+';
                        }else if(i == x1 || i == x2) {
                            g[i][j] = '-';
                        }else if(j == y1 || j == y2) {
                            g[i][j] = '|';
                        }else {
                            g[i][j] = ' ';
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            System.out.print(g[i][j]);
        }
        System.out.println("");
    }
}

class Pair implements Comparable<Pair>{
    int x, y, a, b;
    int index;
    public Pair(int x, int y, int a, int b, int i) {
        this.x = x;
        this.y = y;
        this.a = a;
        this.b = b;
        this.index = i;
    }

    public void close() {
        a = 0;
        b = 0;
    }

    public void resize(int p, int q, int i) {
        a = p;
        b = q;
        index = i;
    }

    public void move(int p, int q, int i) {
        x += p;
        y += q;
        index = i;
    }

    public void active(int i) {
        index = i;
    }

    @Override
    public int compareTo(Pair o) {
        return Integer.compare(this.index, o.index);
    }
}

```

5. 迷宫

```

import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);
    static int N = 2010,n,m;
    static long ans = 0;

```

```

static int[][] trans = new int[N][N];
static int[][] d = new int[N][N];
static int[] dx = new int[] {1,0,-1,0};
static int[] dy = new int[] {0,1,0,-1};
public static void bfs() {
    for(int i = 0; i < N; i++) Arrays.fill(d[i],-1);
    Queue<Pair> q = new LinkedList<>();
    d[n-1][n-1] = 0;
    q.add(new Pair(n-1, n-1));
    while(!q.isEmpty()) {
        Pair t = q.poll();
        if (trans[t.x][t.y] != -1) {
            int tx = trans[t.x][t.y] / n;
            int ty = trans[t.x][t.y] % n;
            if (d[tx][ty] == -1) {
                d[tx][ty] = d[t.x][t.y] + 1;
                q.add(new Pair(tx, ty));
            }
        }
        for(int i = 0; i < 4; i++) {
            int tx = t.x + dx[i];
            int ty = t.y + dy[i];
            if (tx >= 0 && tx < n && ty >= 0 && ty < n && d[tx][ty] == -1) {
                d[tx][ty] = d[t.x][t.y] + 1;
                q.add(new Pair(tx, ty));
            }
        }
    }
}

}

public static void main(String[] args) {
    n = sc.nextInt();
    m = sc.nextInt();
    for(int i = 0; i < N; i++) Arrays.fill(trans[i], -1);
    for(int i = 0; i < m; i++) {
        int x1 = sc.nextInt() - 1;
        int y1 = sc.nextInt() - 1;
        int x2 = sc.nextInt() - 1;
        int y2 = sc.nextInt() - 1;
        trans[x1][y1] = x2 * n + y2;
        trans[x2][y2] = x1 * n + y1;
    }

    bfs();
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            ans += d[i][j];
        }
    }
    System.out.printf("%.2f", 1.0 * ans / (n * n));

//
    System.out.println(bfs(1, 0));
}

}

class Pair {
    int x;
    int y;
    public Pair(int x, int y) {

```

```

        this.x = x;
        this.y = y;
    }
}

```

6. 小球称重

```

import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);
    static HashSet<Integer> set1 = new HashSet<>();
    static HashSet<Integer> set2 = new HashSet<>();
    static int n,m;
    public static void main(String[] args) {
        n = sc.nextInt();
        m = sc.nextInt();

        while(m-->0) {
            int k = sc.nextInt();
            ArrayList<Integer> left = new ArrayList<>();
            ArrayList<Integer> right = new ArrayList<>();
            for(int i = 0; i < k; i++) left.add(sc.nextInt());
            for(int i = 0; i < k; i++) right.add(sc.nextInt());
            String op = sc.next();
            if (op.equals("<")) {
                if (set1.isEmpty()) set1.addAll(left);
                else set1.retainAll(left); // 交集
                set2.addAll(right);
            } else if (op.equals(">")) {
                for(int i = 0; i < k; i++) {
                    if(set1.isEmpty()) set1.addAll(right);
                    else set1.retainAll(right);
                    set2.addAll(left);
                }
            } else {
                set2.addAll(left);
                set2.addAll(right);
            }
        }

        set1.removeAll(set2);
        if (set1.size() == 0) System.out.println(n - set2.size());
        else System.out.println(set1.size());
    }
}

```

7. 背包和魔法

```

import java.util.*;

public class Main {

```

```

static final Scanner sc = new Scanner(System.in);
static int N = 10010, M = 2010, n, m, k;
static int[][] f = new int[N][2];
static int[] w = new int[M];
static int[] v = new int[M];
public static void main(String[] args) {
    n = sc.nextInt();
    m = sc.nextInt();
    k = sc.nextInt();
    for(int i = 0; i < n; i++) {
        w[i] = sc.nextInt();
        v[i] = sc.nextInt();
    }

    for(int i = 0; i < n; i++) {
        for(int j = m; j >= w[i]; j--) {
            if(j >= w[i]) {
                f[j][0] = Math.max(f[j][0], f[j-w[i]][0] + v[i]);
                f[j][1] = Math.max(f[j][1], f[j-w[i]][1] + v[i]);
            }
            if (j >= w[i] + k) {
                f[j][1] = Math.max(f[j][1], f[j-w[i]-k][0] + 2 * v[i]);
            }
        }
    }

    System.out.println(Math.max(f[m][0], f[m][1]));
}
}

```

8.修路

```

import java.util.*;

public class Main {
    static final Scanner sc = new Scanner(System.in);
    static int N = 2010, d, n, m;
    static double inf = Double.MAX_VALUE;
    static int[] a = new int[N], b = new int[N];
    static double[][][] dp = new double[N][N][2];
    public static void main(String[] args) {
        n = sc.nextInt();
        m = sc.nextInt();
        d = sc.nextInt();

        for(int i = 1; i <= n; i++) a[i] = sc.nextInt();
        for(int i = 1; i <= m; i++) b[i] = sc.nextInt();
        Arrays.sort(a, 1, n+1);
        Arrays.sort(b, 1, m+1);

        for(int i = 1; i <= n; i++) {
            dp[i][0][0] = a[i]; //只走了A线, 最终停在A
            dp[i][0][1] = inf; // 只走了A线, 但是最后停在
        }

        for(int j = 1; j <= m; j++) {

```

```

        dp[0][j][0] = inf; //矛盾, 同理
        dp[0][j][1] = get(0, b[1]) + b[j] - b[1]; //从A线起点走
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            dp[i][j][0] = Math.min(dp[i-1][j][0]+a[i]-a[i-1], dp[i-1][j][1] +
get(a[i], b[j]));
            dp[i][j][1] = Math.min(dp[i][j-1][1]+b[j]-b[j-1], dp[i][j-1][0] +
get(b[j], a[i]));
        }
    }

    System.out.printf("%.2f",Math.min(dp[n][m][0], dp[n][m][1]));
//
    }

    public static double get(int a, int b) {
        return Math.sqrt(d * d + Math.pow((a-b), 2));
    }
}

```